

AI-Driven Kubernetes Ingress Security: A JaiClaw Client Engagement

Abstract

A client operating production Kubernetes workloads needed real-time ingress threat detection with a management interface accessible to security analysts—not just Kubernetes administrators. We delivered an AI-powered security operator built on the JaiClaw AI orchestration framework. The solution watches NGINX Ingress traffic in real-time, detects and blocks threats autonomously, and exposes all operational controls through natural-language interfaces via Telegram and CLI. A JaiClaw-scheduled cron job generates and emails comprehensive daily threat reports without any custom scheduling infrastructure. This paper describes the architecture, the role of JaiClaw as the conversational operations layer, and the outcomes achieved.

Introduction: The Problem

Kubernetes ingress security demands constant vigilance. NGINX Ingress Controllers serve as the front door to cluster workloads, processing every inbound HTTP request. Detecting and blocking malicious traffic—vulnerability scanners, credential stuffing attacks, path traversal attempts—requires continuous log analysis, rapid response, and ongoing maintenance of deny lists.

Traditional approaches rely on manual log inspection, threshold-based alerting scripts, and `kubectl` commands to create or modify blocking rules. This workflow is slow: by the time an operator notices an attack pattern, crafts a YAML manifest, and applies it, the attacker may have already moved on—or succeeded. It also demands deep Kubernetes expertise from the security team, creating a bottleneck where only cluster administrators can respond to threats.

The client's security analysts understood threats but lacked Kubernetes fluency. They needed a system where responding to an attack was as simple as typing "block that IP for 24 hours" in a chat window.

The Engagement

The client operates multiple Kubernetes-hosted services behind NGINX Ingress Controllers. Their security team monitored access logs manually and maintained static IP blocklists updated on an ad-hoc basis. Response times to active scanning campaigns ranged from hours to days.

We were engaged to deliver a solution that would: detect threats in real-time from NGINX access logs, enforce blocks autonomously for high-severity threats, provide conversational management for security analysts via Telegram, and produce automated daily status reports. We built the solution on JaiClaw, leveraging its tool orchestration, channel integration, and cron scheduling capabilities.

Architecture Overview

The solution is a Kubernetes Operator—a custom controller that extends the Kubernetes API with domain-specific resources and reconciliation logic.

Threat Detection Pipeline

- Log Streaming** — A watcher component streams access logs from NGINX Ingress pods in real-time, parsing each log entry into structured records (source IP, path, user agent, status code, timestamp).
- Pattern Detection** — Incoming requests are evaluated against 15 known scanner user-agent signatures (zgrab, sqlmap, nikto, nmap, masscan, dirbuster, gobuster, wpscan, acunetix, nessus, openvas, burpsuite, and others) and 28 vulnerability probe paths (`/.env`, `/.git/config`, `/wp-login.php`, `/actuator`, `/aws/credentials`, `/phpmyadmin`, and more). Additional detection covers path traversal sequences and SQL injection patterns.

- 3. **Rate Tracking** — A sliding-window rate tracker monitors requests-per-IP using a configurable time window (default: 1 minute). IPs exceeding the threshold (default: 100 requests/minute) trigger rate-based threat reports. The implementation supports both in-memory tracking (single-instance) and Redis-backed distributed tracking (multi-replica deployments).
- 4. **AI Classification** — For ambiguous patterns that don't match known signatures, the system invokes an LLM (Claude via Spring AI) to classify the threat, providing context about the request patterns, paths targeted, and user agents observed.

State Model: Custom Resources

The solution defines three Kubernetes Custom Resources (API group: `sentinel.taptech.net/v1`):

- **IPBlockRule** — Represents an IP or CIDR range to block, with duration, threat level, reason, and source attribution. Status tracks enforcement state and expiry.
- **ThreatReport** — Records a detected threat with source IP, category (SCANNER, CVE_PROBE, SQL_INJECTION, BRUTE_FORCE, PATH_TRAVERSAL, CREDENTIAL_STUFFING, BOT, DDoS), severity level, request count, targeted paths, and optional AI analysis.
- **BlockList** — References an external blocklist URL (e.g., Spamhaus DROP) with refresh interval and parse format, enabling automatic import of community-maintained threat intelligence.

Enforcement

Kubernetes Operator SDK reconcilers watch these CRDs. When an IPBlockRule is created or updated, the reconciler invokes an enforcer that patches the NGINX Ingress ConfigMap. The enforcer maintains a clearly-demarcated section within the NGINX `server-snippet` configuration, inserting `deny` directives for each active rule. Content outside the managed section is preserved, allowing coexistence with manually-configured directives. NGINX automatically reloads when its ConfigMap changes, achieving enforcement without pod restarts.

Expired rules are automatically detected and cleaned up during reconciliation, removing their deny directives from the ConfigMap.

JaiClaw as the Conversational Operations Layer

This is where JaiClaw transforms the solution from a traditional operator into a conversationally-managed security system.

Registered Tools

Six operational tools are registered with JaiClaw's ToolRegistry:

Tool	Function
<code>sentinel_ban_ip</code>	Block an IP/CIDR with specified duration, reason, and threat level
<code>sentinel_unban_ip</code>	Remove a block and update the NGINX deny list
<code>sentinel_list_blocks</code>	List all active IPBlockRule CRDs with details
<code>sentinel_threat_report</code>	Query recent threats, optionally filtered by IP
<code>sentinel_generate_report</code>	Generate and email a comprehensive status report
<code>send_email_report</code>	Send a formatted report via SMTP

Natural-Language Operations

With these tools registered, operators interact with the security system through plain English via Telegram or the Spring Shell CLI. The LLM agent interprets intent, selects the appropriate tool, and executes it.

Example interactions:

- *"Ban 203.0.113.50 for 7 days — it's been probing our admin endpoints all morning"* → Agent calls `sentinel_ban_ip` with the IP, duration "7d", reason as stated, threat level HIGH
- *"Show me what's been blocked in the last hour"* → Agent calls `sentinel_list_blocks` and summarizes active rules
- *"What threats have we seen from the 10.0.0.0/24 range?"* → Agent calls `sentinel_threat_report` filtered by IP prefix, then synthesizes findings
- *"Unban 192.168.1.100 — that was a false positive from our monitoring system"* → Agent calls `sentinel_unban_ip`, which deletes the CRD and triggers reconciliation to remove the deny directive

The agent also provides contextual recommendations. When asked "Should I block this IP?", it can query the threat history, assess the severity, and advise on an appropriate duration—before the operator confirms and it executes.

Replacing kubectl with Conversation

Previously, blocking an IP required:

```
cat <<EOF | kubectl apply -f -
apiVersion: sentinel.taptech.net/v1
kind: IPBlockRule
metadata:
  name: block-203-0-113-50
spec:
  ip: "203.0.113.50"
  reason: "Scanner detected"
  duration: "24h"
  threatLevel: HIGH
EOF
```

Now it requires: *"Ban 203.0.113.50 for 24 hours, scanner detected."*

This eliminates the Kubernetes expertise requirement entirely. Security analysts operate the system using the language they already speak.

Security

The Telegram integration includes built-in security controls provided by JaiClaw:

- **User ID Allowlist** — Only pre-authorized Telegram user IDs can interact with the bot. JaiClaw's `TelegramUserIdFilter` is automatically configured when allowed users are specified.
- **Per-User Rate Limiting** — JaiClaw's `UserRateLimiter` enforces a configurable rate limit (default: 10 requests/minute per user) to prevent abuse.

No custom security code was required—JaiClaw provides these controls declaratively through configuration.

Automated Security Reporting via Cron Jobs

The client needed daily security status reports delivered to their team without manual intervention. JaiClaw's `CronJobManagerService` handles this natively.

Configuration

A single cron job definition schedules the report:

- **Schedule:** `0 6 * * *` (6:00 AM daily, configurable)
- **Timezone:** America/New_York (configurable)
- **Prompt:** "Call sentinel_generate_report to generate and send the daily threat report"

At the scheduled time, JaiClaw triggers an AI agent session with the configured prompt. The agent invokes the report generation tool, which:

1. Queries all ThreatReport CRDs across namespaces
2. Queries all active IPBlockRule CRDs
3. Aggregates statistics: threat counts by level and category, top 10 offending IPs, top 10 targeted paths
4. Identifies blocks created in the last 24 hours
5. Generates a dual-format report (responsive HTML with inline styling + plain text fallback)
6. Emails the report to configured recipients via SMTP

Report Contents

The generated report includes:

- **Executive summary** with system health status and headline metrics
- **Threat level breakdown** — counts at CRITICAL, HIGH, MEDIUM, LOW with color indicators
- **Top threat categories** — ranked by frequency
- **Top offending IPs and targeted paths** — identifying the most active threats
- **Detailed threat table** — per-threat rows with status badges (BLOCKED/UNBLOCKED), request counts, and observed user agents
- **Quick action commands** — copy-paste Telegram commands for any unblocked IPs, enabling immediate response
- **Active blocks summary** — all currently enforced rules with expiry information

No Custom Infrastructure

The critical point: no external cron daemon, no Kubernetes CronJob manifest, no custom scheduler. JaiClaw manages the schedule internally, invoking the AI agent at the configured time. The agent uses the same tools available to human operators—the scheduled job is simply an automated conversation.

Event-Driven Alerting

When a new ThreatReport CRD is created (whether from pattern detection, rate threshold breach, or AI classification), the reconciler publishes a Spring application event. JaiClaw's gateway subscribes to these events and pushes real-time Telegram alerts to authorized users.

The alert includes the source IP, threat level, category, and action taken. Operators can immediately respond conversationally:

- *"Tell me more about that IP"* → Agent queries full threat history
- *"Block it permanently"* → Agent executes ban with "permanent" duration
- *"Looks like a false positive, ignore it"* → Operator acknowledges, no action needed

This creates a tight feedback loop: detection → alert → human decision → execution, all within a single chat interface.

Results and Impact

The engagement delivered measurable improvements to the client's security posture:

- **Sub-second detection to enforcement** — High and critical threats trigger automatic IPBlockRule creation and NGINX enforcement within the same reconciliation cycle
- **Zero-kubectl operations** — Security analysts manage the system entirely through Telegram conversation, with no Kubernetes expertise required
- **24/7 autonomous monitoring** — The operator runs continuously, detecting and blocking threats without human intervention for high-severity cases, while maintaining human-in-the-loop for edge cases via conversational alerts
- **Automated reporting** — Daily status reports delivered to the team inbox at 6 AM, generated by the AI agent using the same tools as interactive operations
- **Complete audit trail** — Every block, threat detection, and action is recorded as a Kubernetes Custom Resource, providing native audit history through the K8s API
- **Reduced mean time to respond** — From hours (manual log review → YAML authoring → kubectl apply) to seconds (read alert → type response)

Conclusion

This engagement demonstrates how JaiClaw transforms infrastructure security operations from reactive, expertise-gated CLI work into proactive, conversational AI management. The key architectural insight is that by registering operational tools with JaiClaw's orchestration layer, any system capability becomes accessible through natural language—whether invoked by a human operator in Telegram, a scheduled cron job, or an event-driven alert response.

The pattern is broadly applicable: any system that exposes operational tools can be made conversationally manageable through JaiClaw. The security operator described here is one instance of a general architecture where AI agents bridge the gap between complex infrastructure APIs and the humans who need to operate them.